

M2 Course content

Introduction

- Language reminders:
 - VBA
 - Python
- Classes in VBA
- Classes in Python
- Project: Option pricing with a lattice

7/14/2025 21:50:02

Pricing model landscape

- Here are the three types of pricing models:

	Closed-form formulas	Trees	Monte-Carlo
Examples	Black-Scholes, Heston, SABR	Hull & White	Longstaff-Schwartz
European ex.	Yes	Yes	Yes
American ex.	No	Yes	Hard
Path dependent	No	Hard	Yes
Speed	Instantaneous	Fast	Slow
Precision	Perfect	Error $\geq 1/Nb$ Steps	Error $\geq 1/vNb$ Draws
Memory	Light	Depends	Depends

Lattice node VBA class

- Goal #1, practice PDE and trees:
 - Simulate a stock price in a recombining lattice
 - Price European and American calls and puts
 - Assess convergence with Black-Scholes
- Goal #2, code and use classes:
 - Define classes for the lattice node and tree
 - Build a tree with reconnecting nodes
 - Price options on this lattice

7/14/2025 21:50:02

Stock price behavior

- In the « *Derivatives Pricing and Stochastic Calculus* » course by Elie & Kharroubi we can see for Black & Scholes p. 61:

- **The risky asset.** We suppose that the risky asset S is given by the SDE

$$\begin{cases} dS_t = S_t (\mu dt + \sigma dW_t), & t \in [0, T] \\ S_0 = s_0 \end{cases} \quad (6.2.1)$$

where μ and σ are two constants such that $\sigma > 0$.

- We can use those assumptions for the interest rate and the volatility
- But the dividend is discrete and it is simplistic to define μ as: $\mu = \text{rate} - \text{dividend}$

7/14/2025 21:50:02

Discrete dividends

- Dividends in a near future are known
- Dividends in several years can be approximated as a % of the stock price
- Example with a stock price $S_0 = 100 \text{ €}$
 - First annual dividend = 3 €
 - Second annual dividend = 2 € + 1% of X_{1Y}
 - Third annual dividend = 1.4 € + 1.6% of X_{2Y}
 - Fourth annual dividend = 1 € + 2% of X_{3Y}
 - ...
 - Tenth annual dividend = 3% of X_{10Y}

7/14/2025 21:30:02

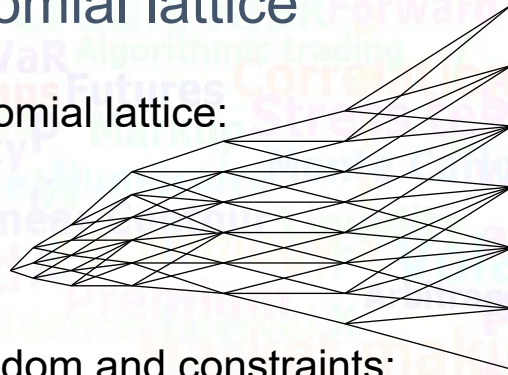
Dividend model

- The dividend is discrete and moves from constant to proportional to the stock price
- Example of possible functional form:
 - $D_t = \rho (S_0 e^{-\lambda(t-t_0)} + S_t (1 - e^{-\lambda(t-t_0)}))$ on ex-div. dates
 - $D_t = 0$ on all other dates
- $dS_t = S_t r dt + S_t \sigma dW_t - D_t$

7/14/2025 21:30:02

Trinomial lattice

- Example of trinomial lattice:



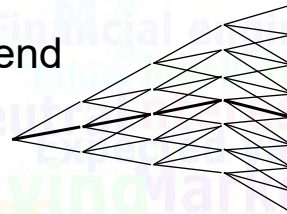
- Degrees of freedom and constraints:

Degrees of freedom	Constraints
Time steps	Match expected value
Nodes values	Match variance
Choice of next middle node	Positive probabilities
Transition probabilities	Sum of probabilities = 1

7/14/2025

Trinomial tree assumptions

- We use the following assumptions:
 - Time steps are all equal: Δt
 - The middle node is equal to the forward price
 - Nodes values are geometric series: $\alpha = S_{up}/S_{mid}$
 - The next middle node is the closest to the forward price
- Example with a dividend on the fifth date:



7/14/2025 21:50:02

Lattice building

- Start from the root on date d_0 : N_{start} with $S_{\text{start}} = s_0$
- Branch to the next central node: N_{mid}

$$S_{\text{mid}} = S_{\text{start}} \exp(r \Delta t) - D_1$$
- Build other nodes on date d_1 : $S_{\text{up}} = S_{\text{mid}} \alpha$
- α is defined by a multiple of the standard deviation over one time step $\approx S_t \sigma \sqrt{\Delta t}$:

$$S_{\text{up}} - S_{\text{mid}} \approx \sqrt{3} \text{StdDev} = \sqrt{3} S_{\text{mid}} \sqrt{e^{\sigma^2 \Delta t} - 1}$$
- Divide by S_{mid} : $\alpha \approx 1 + \sqrt{3} \text{StdDev} / S_{\text{mid}}$
- The actual formula is: $\alpha = e^{\sigma \sqrt{3 \Delta t}}$

7/14/2025 21:50:01 Please note: $\sqrt{3}$ is indicative. $\sqrt{2}$ as well as 2 work too

Probabilities (1)

- For each node N we calculate the probabilities:
 - The sum of the three probabilities is equal to 1:

$$p_{\text{up}} + p_{\text{mid}} + p_{\text{down}} = 1$$

- Matching the expected value means:

$$E_{i+1} = p_{\text{up}} S_{i+1,\text{up}} + p_{\text{mid}} S_{i+1,\text{mid}} + p_{\text{down}} S_{i+1,\text{down}} = E[S_{t_{i+1}} | S_{t_i}]$$

- With $S_{t_{i+1}} = S_{t_i} e^{(r - \frac{1}{2}\sigma^2)\Delta t + \sigma W_{\Delta t}} - D_{t_{i+1}}$

$$E[S_{t_{i+1}} | S_{t_i}] = S_{t_i} e^{r\Delta t} - D_{t_{i+1}}$$

7/14/2025 21:50:02

Probabilities (2)

- Matching the variance on the step after N means:

$$V_{i+1} + E_{i+1}^2 = p_{up} S_{i+1,up}^2 + p_{mid} S_{i+1,mid}^2 + p_{down} S_{i+1,down}^2$$

$$V_{i+1} = E \left[\left(S_{t_{i+1}} - E[S_{t_{i+1}} | S_{t_i}] \right)^2 | S_{t_i} \right]$$

$$\circ V_{i+1} = E \left[S_{t_i}^2 \left(e^{(r - \frac{1}{2}\sigma^2)\Delta t + \sigma W_{\Delta t}} - e^{r\Delta t} \right)^2 | S_{t_i} \right]$$

$$\circ V_{i+1} = S_{t_i}^2 e^{2r\Delta t} E \left[e^{-\sigma^2\Delta t + 2\sigma W_{\Delta t}} - 2e^{-\frac{1}{2}\sigma^2\Delta t + \sigma W_{\Delta t}} + 1 \right]$$

$$\circ V_{i+1} = S_{t_i}^2 e^{2r\Delta t} (e^{\sigma^2\Delta t} - 1)$$

7/14/2025 21:50:02

Probabilities (3)

- The three constraints make a linear system:

$$\circ 1 = p_{up} + p_{mid} + p_{down}$$

$$\circ E_{i+1} = p_{up} S_{i+1,up} + p_{mid} S_{i+1,mid} + p_{down} S_{i+1,down}$$

$$\circ V_{i+1} + E_{i+1}^2 = p_{up} S_{i+1,up}^2 + p_{mid} S_{i+1,mid}^2 + p_{down} S_{i+1,down}^2$$

- That can be written as a matrix product:

$$\begin{pmatrix} 1 & 1 & 1 \\ S_{i+1,up} & S_{i+1,mid} & S_{i+1,down} \\ S_{i+1,up}^2 & S_{i+1,mid}^2 & S_{i+1,down}^2 \end{pmatrix} \begin{pmatrix} p_{up} \\ p_{mid} \\ p_{down} \end{pmatrix} = \begin{pmatrix} 1 \\ E_{i+1} \\ V_{i+1} + E_{i+1}^2 \end{pmatrix}$$

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & S_{i+1,mid} & 0 \\ 0 & 0 & S_{i+1,mid}^2 \end{pmatrix} \begin{pmatrix} 1 & 1 & 1 \\ \alpha & 1 & \alpha^{-1} \\ \alpha^2 & 1 & \alpha^{-2} \end{pmatrix} \begin{pmatrix} p_{up} \\ p_{mid} \\ p_{down} \end{pmatrix} = \begin{pmatrix} 1 \\ E_{i+1} \\ V_{i+1} + E_{i+1}^2 \end{pmatrix}$$

7/14/2025 21:50:02

Probabilities (4)

- The system has a unique solution with $\alpha > 1$ and $S_{i+1} > 0$:

$$\begin{pmatrix} 1 & 1 & 1 \\ \alpha & 1 & \alpha^{-1} \\ \alpha^2 & 1 & \alpha^{-2} \end{pmatrix} \begin{pmatrix} p_{up} \\ p_{mid} \\ p_{down} \end{pmatrix} = \begin{pmatrix} 1 \\ S_{i+1,mid}^{-1} E_{i+1} \\ S_{i+1,mid}^{-2} (V_{i+1} + E_{i+1}^2) \end{pmatrix}$$

- Solve by substitutions:

$$\begin{pmatrix} 1 & 1 & 1 \\ \alpha - 1 & 0 & \alpha^{-1} - 1 \\ \alpha^2 - 1 & 0 & \alpha^{-2} - 1 \end{pmatrix} \begin{pmatrix} p_{up} \\ p_{mid} \\ p_{down} \end{pmatrix} = \begin{pmatrix} 1 \\ S_{i+1,mid}^{-1} E_{i+1} - 1 \\ S_{i+1,mid}^{-2} (V_{i+1} + E_{i+1}^2) - 1 \end{pmatrix}$$

- Subtract $(\alpha + 1)$ times the second line to the third to get:

$$\begin{aligned} & (1 - \alpha)(\alpha^{-2} - 1) p_{down} \\ &= S_{i+1,mid}^{-2} (V_{i+1} + E_{i+1}^2) - 1 - (\alpha + 1)(S_{i+1,mid}^{-1} E_{i+1} - 1) \end{aligned}$$

- p_{down} has no dimension (good!), but is not guaranteed to be in $[0, 1]$

7/14/2025 21:50:02

Probabilities (5)

$$p_{down} = \frac{S_{i+1,mid}^{-2} (V_{i+1} + E_{i+1}^2) - 1 - (\alpha + 1)(S_{i+1,mid}^{-1} E_{i+1} - 1)}{(1 - \alpha)(\alpha^{-2} - 1)}$$

- When there is no dividend, $S_{i+1,mid} = E_{i+1}$ and we have:

$$p_{down} = \frac{S_{i+1,mid}^{-2} V_{i+1}}{(1 - \alpha)(\alpha^{-2} - 1)} = \frac{e^{\sigma^2 \Delta t} - 1}{(1 - \alpha)(\alpha^{-2} - 1)}$$

- Use the second row to solve p_{up} :

$$(\alpha - 1)p_{up} + (\alpha^{-1} - 1)p_{down} = S_{i+1,mid}^{-1} E_{i+1} - 1$$

- Without div: $p_{up} = \frac{p_{down}}{\alpha}$

- Use the sum to solve p_{mid}

European option pricing

- Build a lattice with N steps and $t_N = T$, the option maturity
- Then value the payoff on each node on T
 - On ITM nodes the payoff is for a call: $S_N - K$
 - On OTM nodes the payoff is equal to 0
- Propagate the net future value on t_{N-1} nodes:
 - NFV_{N-1} = sum of probability $\times$ next node value $\times$ DF
 - DF is the discount factor = $\exp(-r \Delta t)$
- NFV_0 = NPV is the net present value of the option

7/14/2025 21:50:02

American option pricing

- Same as European options except for the early exercise
- At any time before the maturity the option holder can exercise the option
- On each interim node the value is the maximum of two quantities:
 - Hold: value = discounted value like for European case
 - Exercise: value = $\max(S_i - K, 0)$ for a call

7/14/2025 21:50:02

Assignment (1)

- Build a trinomial lattice and an option pricer
 - Start with few steps, no dividend, European exercise
 - Try to have as much code as possible in the classes
 - Check that the price converges to Black formula
 - Describe the gap as a function of the number of steps
 - Describe the gap as a function of the strike
- Add the American vs. European exercise
 - Study the difference of price American – European
 - Observe the effect of the interest rate (>0 or <0)
 - Describe the cases where the two prices are equal

7/14/2025 21:50:02

Assignment (2)

- Add the dividend
 - With interest rates = 0, see the impact on the call and the put
- Other possible extensions:
 - Graph tree
 - Option value
 - Exercise frontier
 - Search the limit of number of steps (time, memory)
 - Try to remove nodes with very low probability
 - Describe the effect on run time for 1000 steps
 - Extend the number of steps
 - Set the number of steps from a user-defined precision

7/14/2025 21:50:02

Assignment (3)

- Recursive pricing:
 - Either locate the code in the Model/Tree class:

```
' Call recursive pricing
Let OptionPricerRecVBA = model.Price(opt, t.Root)
```

- Or in the Node class:

```
optionPrice: float = model.tree.root.priceRecursive(opt)
```

- Choose, in VBA and in Python, the design you prefer

7/14/2025 21:50:02

Assignment (4)

- Derivative:

- Simple design in VBA:

```
Let shift = startPrice * 0.01
Let priceUp = OptionPricerRecVBA(nSteps, startDate, matDate, _
    startPrice + shift, rate, volat, div, exDate, optType, exer, k, False)
Let priceDown = OptionPricerRecVBA(nSteps, startDate, matDate, _
    startPrice - shift, rate, volat, div, exDate, optType, exer, k, False)
Let OptionPricerRecVBADelta = (priceUp - priceDown) / (2 * shift)
End Function
```

- In Python we can pass a function as a parameter:

```
class OneDimDerivative:

    def __init__(self, function: Callable[[object, float], float],
                 other_parameters: object, shift: float = 1):
        self.f: Callable[[object, float], float] = function
        self.param: object = other_parameters
        self.shift: float = shift

    2 usages
    def first(self, x: float) -> float:
        return (self.f(self.param, x + self.shift)
                - self.f(self.param, x - self.shift)) / (2 * self.shift)

def OptionDeltaTreeBackward(market_range: numpy.array, pricer_range: numpy.array, option_range: numpy.array) -> float:
    return OneDimDerivative(cast((object, float), _PriceTreeBackward_OneDimPrice),
                             OptionPricingParam(market_range, pricer_range, option_range),
                             market_range[0] * OptionPricingParam.UND_SHIFT).first(market_range[0])
```

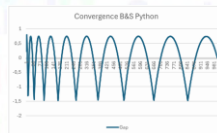
Assignment (5)

- Other trick to calculate delta and gamma:
 - Add up and down nodes around the root
 - Calculate the option prices in the whole tree
 - Deduce the delta from the option and underlying prices on the up and down nodes
 - Ditto for the gamma from the three nodes
- There is no similar variation for vega and vomma

7/14/2025 21:50:02

Assignment (6)

- Report: short and informative
 - When you write 10 pages, I read 250
- Express:
 - What you did
 - Your noticeable findings
 - Your difficulties...
 - ...and how you overcame them (when you did)
- I love nice graphs



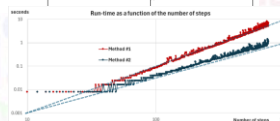
Edi VERBOVCI
Timothée DANGLETERRE

Rapport : étude de cas - Pricing par la méthode de l'arbre trinomial

1. Facultés du code :

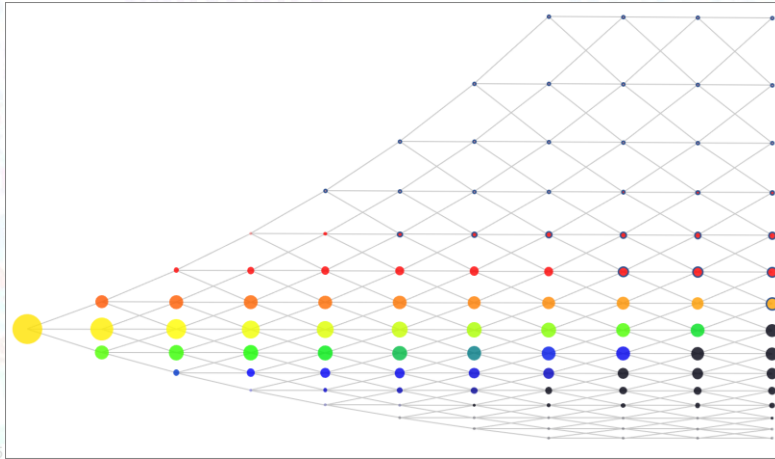
Dans le cadre de ce projet, nous avons réalisé en Python et VBA un code orienté objet permettant de construire un arbre trinomial pour valoriser une option Américaine / Européenne en tenant compte d'un éventuel versement de dividende. Les deux codes sont proches mais n'ont pas tout à fait les mêmes fonctionnalités. Les applications possibles sont résumées dans le tableau ci-dessous :

	VBA	Python
Construction de l'arbre trinomial (+1000 pas)	X	X
Méthode de pricing	Récurrent & Backward	Backward
Représentation graphique de l'arbre	X	
Pricing au niveau de l'arbre	X	X
Calcul des Grecques par la méthode de Black & Scholes	X	
Calcul des Grecques au premier ordre par différence finie		X



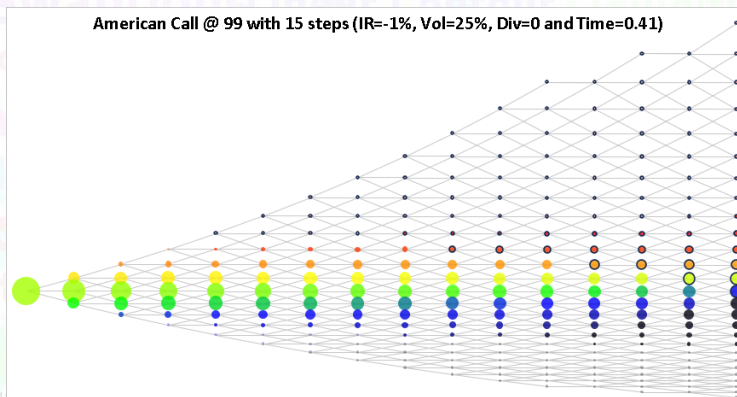
Example of tree with 10 steps

- 1Y ATM American call (vol=30%, int. rate=-1%)



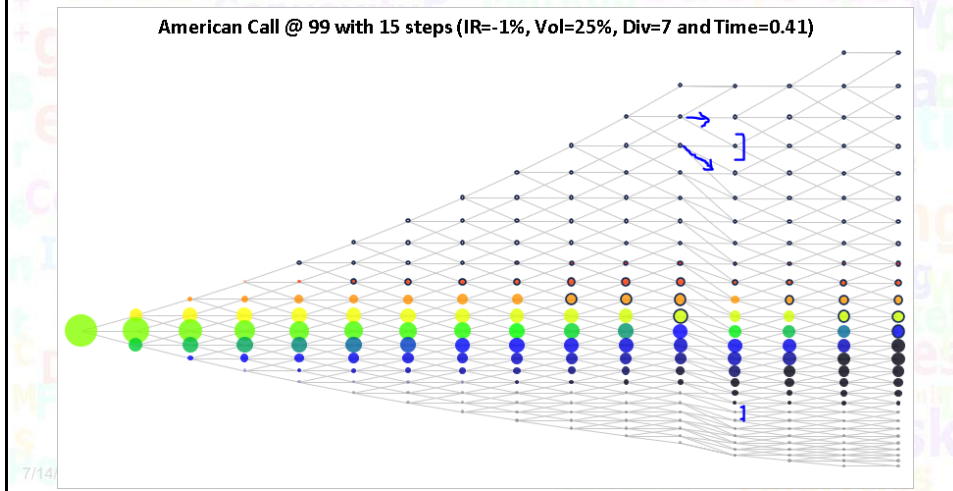
Effect of discrete dividends (1)

- In general, the middle nodes are aligned:
 - The j^{th} node above the trunk on date i connects with the j^{th} node above the trunk on date $i+1$ as next mid



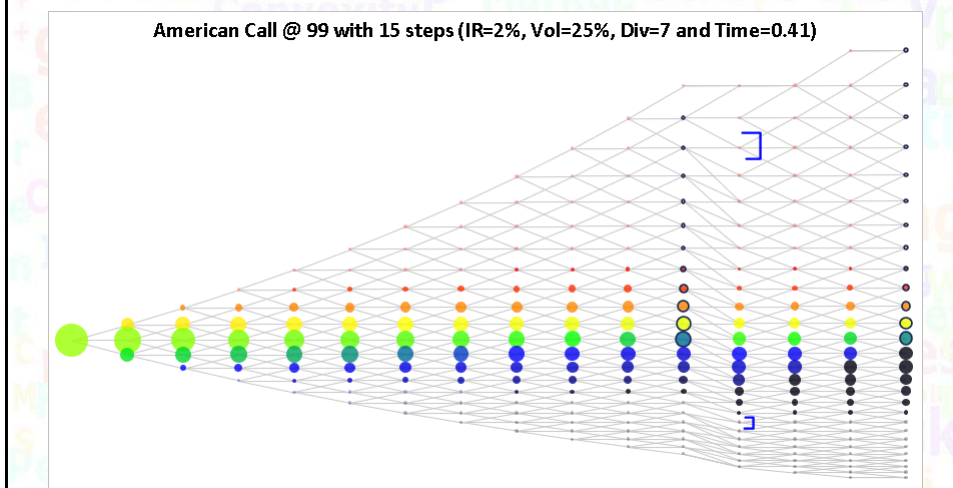
Effect of discrete dividends (2)

- But that doesn't hold with dividends:



Effect of discrete dividends (3)

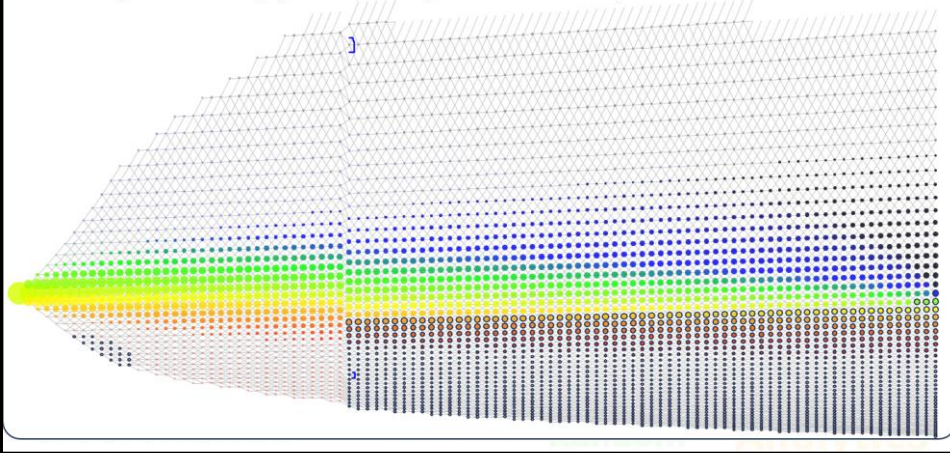
- Same with positive interest rate:



Effect of discrete dividends (4)

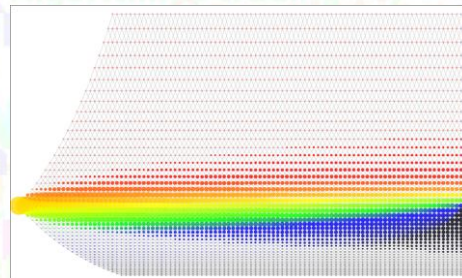
- Other example with a put:

American Put @ 101 with 100 steps (IR=15.0%, Vol=30%, Div=3 EUR and Time=0.8)

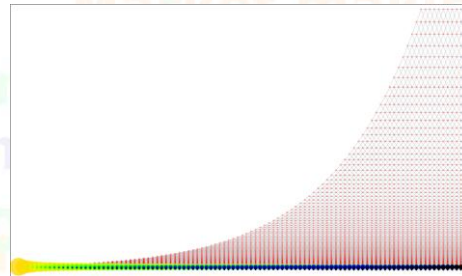


Example of tree with 100 steps

- With prices between 30 and 300:



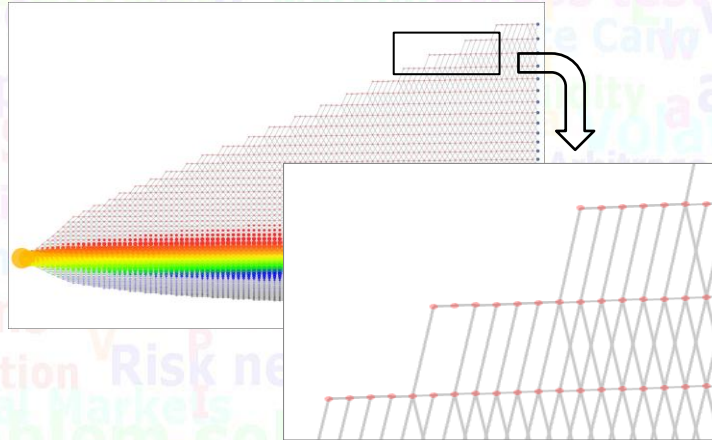
- ...and between 1 and 10,000:



7/14/2025 21:50:02

Tree with side connections cut

- Tree with mid connection below $1E-7$:



7/14/2025 21:50:02

Tree "pruning" (1)

- It consists in not allocating nodes which probability is too small
- A typical design consists in branching to the middle node only, with a probability = 1
- The "monomial" branching can be triggered by one of two designs:
 1. by the probability to reach a node, calculated from the root
 2. or by a number of standard deviations of the node from the middle one

7/14/2025 21:50:02

Tree "pruning" (2)

- We can calculate the probability to reach a node by:
 - starting with the root node with probability = 100%
 - any other node has a probability equal to the sum of previous nodes connected to it x transition probability
 - Example: the top node has a probability equal to the top node on the previous date x transition probability
 - If the move up probability is, say, 0.18, the probability to be on the top node decreases like 0.18^i , with i the number of the column date. After 9 steps the top node probability is close to 2.10^{-7} .

7/14/2025 21:50:02

Tree "pruning" (3)

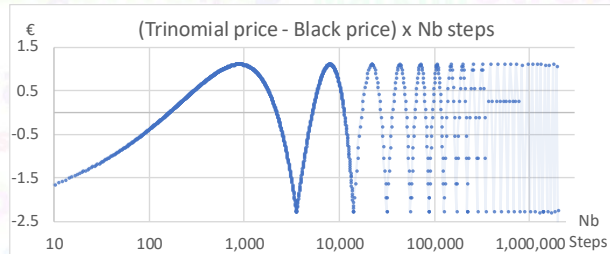
- We can also deduce a node probability by its number of standard deviations from the middle:
 - The process on date i is: $\frac{S_{t_i}}{S_{forward_i}} = e^{\sigma W_{i\Delta t} - \frac{1}{2}\sigma^2\Delta t}$
 - $\ln\left(\frac{S_{t_i}}{S_{forward_i}}\right)$ standard deviation = $\sigma\sqrt{i\Delta t}$
 - The relative space between nodes is $\sigma\sqrt{3\Delta t}$
 - If we allocate nodes over 4 standard deviations we need k nodes on either sides:

$$k\sigma\sqrt{3\Delta t} \geq 4\sigma\sqrt{i\Delta t}, \text{ which simplifies into: } k \geq 4\sqrt{\frac{i}{3}}$$

7/14/2025 21:50:02

Calculate number of steps (1)

- (Tree – BS) x NbSteps gap lies in a tunnel

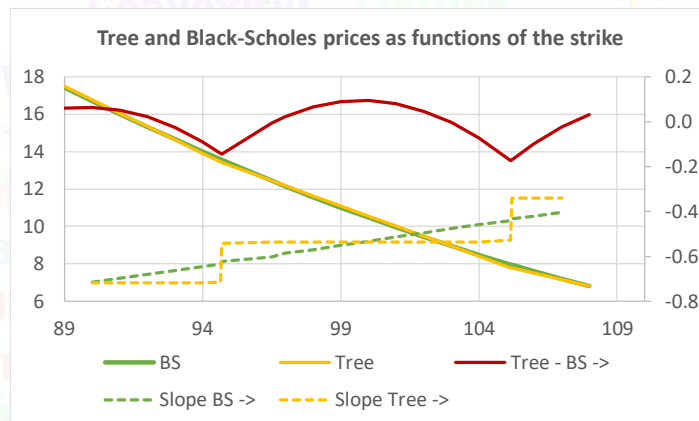


- $\Rightarrow \text{Gap} = f(S_0, \sigma, K, T, r) / \text{NbSteps}$
- $f(S_0, \sigma, K, T, r)$ is a function of other parameters: vol, strike, maturity, etc.

7/14/2025 21:50:02

Calculate number of steps (2)

- Tree vs. Black Price function of K (10 steps):



7/14/2025 21:50:02

Calculate number of steps (3)

- Idea: consider the ATM forward option
- Gap $\approx$ lost value of node where $S = K$
- Value of that node with BS:

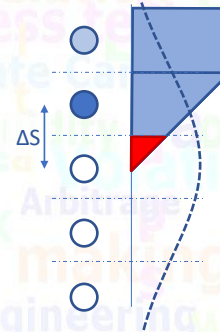
$$\text{Red triangle} \times \text{density} = \frac{1}{2} \left(\frac{\Delta S}{2} \right)^2 \frac{1}{\sqrt{2\pi} \Sigma}$$

$$\Delta S = Fwd(T) (\alpha - 1)$$

$$\Delta S = (S_0 e^{rT} - D) (e^{r\Delta t + \sigma\sqrt{3\Delta t}} - 1)$$

- Σ is the standard deviation of the stock distribution on the maturity date T

$$\Sigma = S_0 e^{rT} \sqrt{e^{\sigma^2 T} - 1}$$



7/14/2025 21:50:02

Calculate number of steps (4)

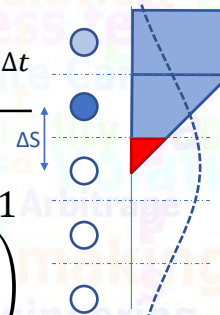
- Gap function of Δt (i.e. of NbSteps) is:

$$Gap(NbSteps) \approx \frac{3 S_0 (e^{\sigma^2 \Delta t} - 1) e^{2r\Delta t}}{8\sqrt{2\pi} \sqrt{e^{\sigma^2 T} - 1}}$$

- The main dependency on Δt is in $e^{\sigma^2 \Delta t} - 1$

$$\Delta t \approx \frac{1}{\sigma^2} \ln \left(1 + \frac{8\sqrt{2\pi} Gap \sqrt{e^{\sigma^2 T} - 1}}{3 S_0 \sigma^2 T} \right)$$

$$NbSteps = \frac{T}{\Delta t} \approx \frac{T}{\frac{1}{\sigma^2} \ln \left(1 + \frac{8\sqrt{2\pi} Gap \sqrt{e^{\sigma^2 T} - 1}}{3 S_0 \sigma^2 T} \right)}$$



7/14/2025 21:50:02

Calculate number of steps (5)

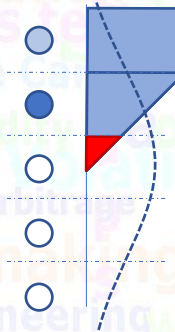
Remarks:

- $\frac{Gap}{S_0}$ is a relative precision on the stock price
- For small gaps, $NbSteps = T / \Delta t$ is inversely proportional to the gap (and the gap to $NbSteps$)
- Simplified formulas:

$$Gap \approx \frac{3 S_0}{8\sqrt{2\pi}} \frac{\sigma^2 \Delta t}{\sqrt{e^{\sigma^2 T} - 1}} \quad \Delta t \approx \frac{8\sqrt{2\pi}}{3} \frac{Gap}{S_0} \frac{\sqrt{e^{\sigma^2 T} - 1}}{\sigma^2}$$

$$NbSteps = \frac{T}{\Delta t} \approx \frac{3}{8\sqrt{2\pi}} \frac{S_0}{Gap} \frac{\sigma^2 T}{\sqrt{e^{\sigma^2 T} - 1}}$$

7/14/2025 21:50:02



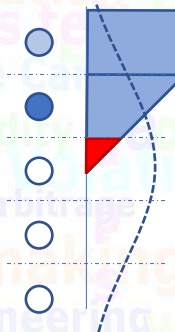
Calculate number of steps (6)

Remarks:

- With $T = 1$ year, $\sigma = 30\%$, we have:

- $\frac{Gap}{S_0} \approx \frac{1}{N} \frac{3}{8\sqrt{2\pi}} \frac{0.3^2}{\sqrt{e^{0.3^2} - 1}}$
- $N=1: \frac{Gap}{S_0} = 4\%$
- $N=10: \frac{Gap}{S_0} = 0.4\%$
- $N=100: \frac{Gap}{S_0} = 0.04\%$
- $N=1000: \frac{Gap}{S_0} = 0.004\%$

7/14/2025 21:50:02



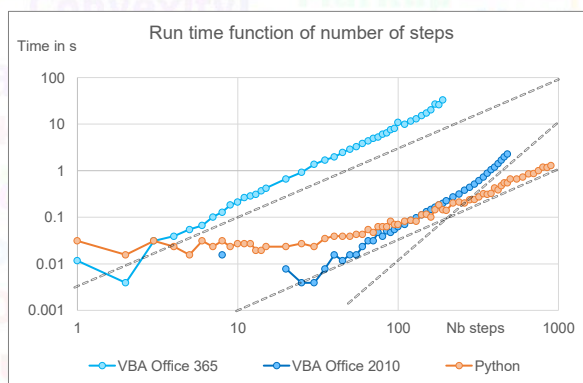
How to check prices?

- You can check several results:
 - European options without dividends compared to Black-Scholes prices
 - European calls with strike = 0, compared with the forward price on maturity, including with dividends
 - American options are harder to test, but the risk of error is lower (very little code is involved) and you can check the exercise frontier

27-Nov-19 21:50:02

Performance: VBA vs. Python

- VBA performance has been poor in 365 version:

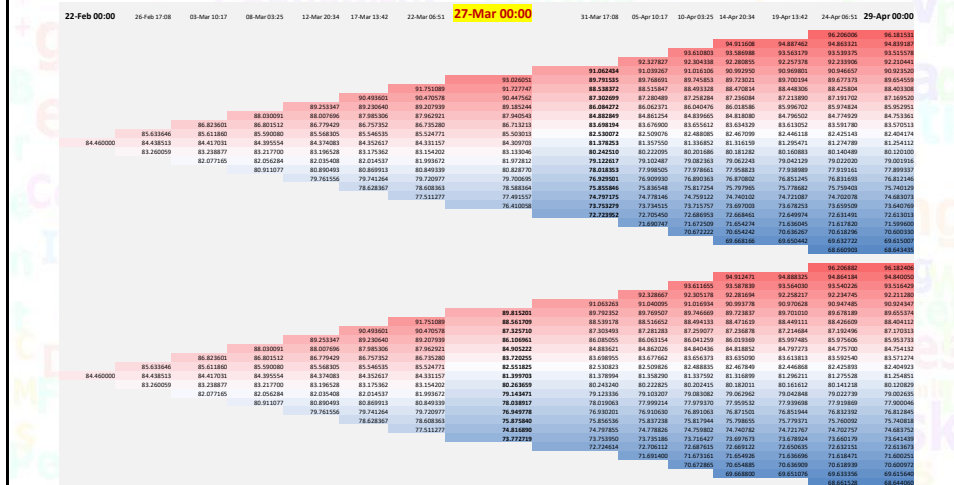


- Price differ by 2.10⁻⁸, for a put very OTM
- Print underlying:
- What happens?

	Start date	22-Feb-22
Market	Start price	84.46 €
	Interest rate	-1.97%
	Volatility	7.14%
	Dividend	2.91 €
	Div ex-date	27-Mar-22
	Maturity	29-Apr-22
Trade	Type	Put
	Exercise	American
	Strike	70.14 €
Model	Nb steps	14

The dividend periods differ

- The dividend is not paid on the same period!



Equality between dates (1)

- Initial date + n x time step can seem different from the ex-dividend date
- Solution: define the equality with a tolerance

' returns True if the div falls between the current date (excluded) and the next one (included)

Public Function IsDivAfterThisDate() As Boolean

Dim divDate As Date

Let divDate = Me.TheTree.Market.DivExDate

Let IsDivAfterThisDate = Not (Me.AreEqualDates(Me.ColumnDate, divDate)) _

And Me.ColumnDate < divDate _

And (divDate < Me.NextDate() Or Me.AreEqualDates(divDate, Me.NextDate()))

End Function

' returns True if the two dates are closer than a small fraction of time

Function AreEqualDates(ByVal d1 As Date, ByVal d2 As Date) As Boolean

Let AreEqualDates = Abs(d1 - d2) < 1 / Me.TheTree.nbSteps / 1000

End Function

Equality between dates (2)

- In Python, as a member in TruncNode:

```
class TruncNode(Node):
    # constructor with reference to the Node constructor
    def __init__(self, precNode, colDate, price, tree):
        Node.__init__(self, price, self, tree)
        self.precMid: TruncNode = precNode
        self.columnDate = colDate
        self.nextMid: TruncNode = None
        exDivDate = self.tree.mkt.divExDate
        # means self.columnDate < exDivDate <= self.nextDate()
        self.isDivInTheFollowingPeriod = not(self.areSameDates(self.columnDate, exDivDate)) \
            and self.columnDate < exDivDate \
            and (exDivDate < self.nextDate() or self.areSameDates(exDivDate, self.nextDate()))

    # this equality between dates avoids small numerical errors that make dates look different
    def areSameDates(self, d1, d2):
        return abs(d1 - d2) < datetime.timedelta(days=1) / self.tree.nbSteps / 1000
```

Price precision: lower in Python

- The largest gaps between VBA and Python are around 1.10^{-11} € on stock prices around 100 €
- Observing the transition probabilities (without dividend), we can see that Python values are 1000 times more instable with low volatilities:

VBA, down probabilities

	0.1667410088542450	0.1667410088542450
0.1667410088543840	0.1667410088543840	0.1667410088543840
	0.1667410088543860	0.1667410088543130
		0.1667410088542420

Python

Start date	08-Jun-22
Start price	112.53
Interest rate	2.00%
Volatility	0.44%
Dividend	0.00
Div ex-date	19-Jun-22
Maturity	08-Jul-22
Type	Call
Exercise	European
Strike	107.44
Nb steps	6

- Why?!

	0.1667410088544570	0.1667410087146660
0.1667410088543950	0.1667410088543950	0.1667410088543950
	0.1667410088544570	0.1667410085750610
		0.1667410088542700

Analysis

- Underlying prices differ only by the last 2 digits
- The same for forward rates
- Variances are all exactly equal
- In the debugger, the only visible gap is between the **forward** and the value on the **next node**, just on the last digit
- Is it enough?

```

fwd = (float) 112.52211692719607
nextN = (Node) <node.Node object at 0x000001B1EC563850>
nextMidPr = (float) 112.52211692719605
self = (Node) <node.Node object at 0x000001B1EC13D6A0>
downNode = (Node) <node.Node object at 0x000001B1EC5C26A0>
hasOptionValue = (bool) False
nextDown = (Node) <node.Node object at 0x000001B1ED9F3B80>
nextMid = (Node) <node.Node object at 0x000001B1EC563850>
nextUp = (TruncNode) <node.TruncNode object at 0x000001B1EC6085E0>
nodeProba = (float) 0.22232133775183982
optionValue = (float) -1.0
probaDown = (float) 0.1667410085750615
probaMid = (float) 0.0
probaUp = (float) 0.16659234635228332
tree = (Tree) <tree.Tree object at 0x000001B1E3968580>
truncNode = (TruncNode) <node.TruncNode object at 0x000001B1E39B6BE0>
underlyingPrice = (float) 112.49129317258068

```

Detailed calculation

- The down probability has the following code:

```

self.probaDown = ((variance + fwd * fwd) / (nextMidPr * nextMidPr) - 1
- (a + 1) * (fwd / nextMidPr - 1)) / ((1 - a) * (1 / (a * a) - 1))

```

$$p_{down} = \frac{S_{i+1,j'}^{-2}(V_{i+1,j} + E_{i+1,j}^2) - 1 - (\alpha + 1)(S_{i+1,j'}^{-1}E_{i+1,j} - 1)}{(1 - \alpha)(\alpha^{-2} - 1)}$$

- The numerator is $2.7 \cdot 10^{-7}$ and the denominator $1.6 \cdot 10^{-6}$. Therefore a small error in the numerator is magnified in p_{down} .
- The solution is to use the simplified formula when there is no dividend (store 3 values in tree)

Other precision points

- It also helps to rephrase the math formula:

$$\frac{V_{i+1,j} + E_{i+1,j}^2}{S_{i+1,j'}^2} - 1$$

- It is less precise when the next node and forward values are close than this variation:

$$\frac{V_{i+1,j} + (E_{i+1,j} + S_{i+1,j'})(E_{i+1,j} - S_{i+1,j'})}{S_{i+1,j'}^2}$$

- Ditto for the other term:

$$\frac{E_{i+1,j}}{S_{i+1,j'}} - 1 = \frac{E_{i+1,j} - S_{i+1,j'}}{S_{i+1,j'}}$$

Organization

- For questions, please use Moodle **Forum**.
- Please post your spreadsheet on Moodle before Sunday 3rd of November
- Please include your name in the files, like this:
 - Folder_Ann-M_Julien-P...
 - Anna-M_Julien-P_...xlsm
 - Anna-M_Julien-P_...: Word or pdf file
 - main.py,
 - tree.py, etc.